

M2 - Architecture et Programmation d'accélérateurs Matériels.

(APM 2024-2025)

TP1

Optimisation des accès mémoire
Utilisation de la mémoire partagée



julien.jaeger@cea.fr
lucas.vanlanker@cea.fr

L'objectif de ce TP est de comprendre les différentes mémoires existantes d'un GPU et leur utilisation sur un calcul classique d'algèbre linéaire : le produit matriciel. Nous allons voir à travers cet exemple comment utiliser la mémoire globale, la mémoire partagée et la mémoire unifiée.

Les objectifs de ce TP sont :

- Maîtriser la chaîne de compilation CUDA
- Compréhension du concept de global memory coalescing GPU,
- Optimisation des performances avec l'utilisation de la mémoire partagée.

Ce TP est à faire sur deux séances.

I Chaîne de compilation

Pour observer certaines étapes du processus de compilation, compiler le fichier *tp1_check.cu* avec la commande :

```
nvcc -keep -keep-dir CUDAIMG -c tp1_check.cu
```

(il faudra avoir créé le répertoire CUDAIMG avant de lancer la commande).

Des fichiers intermédiaires sont créés dans le répertoire CUDAIMG.

Q.1: Quel fichier correspond au PTX ? Lequel correspond au binaire CUDA ?

Correction

Le fichier *.ptx* correspond à l'assembleur PTX, le langage intermédiaire entre CUDA et le binaire CUBIN. Le fichier *...sm_xx.cubin* correspond à la traduction de cet assembleur en binaire CUDA (CUBIN).

Q.2: Quelles parties des sources de *tp1_check.cu* ont été conservées pour être compilées en PTX ? **Remarque :** Vous pouvez lire ce paragraphe de la documentation CUDA pour en apprendre plus sur la chaîne de compilation CUDA.

Correction

Le compilateur nvcc sépare les parties host et device du code. Seule la partie device, donc les kernels sont traduits en assembleur PTX. Le reste du code a été transmis au compilateur C/C++ de l'hôte (par exemple g++).

Q.3: À quoi correspond le suffixe *...sm_xx.cubin* (par exemple sm_30 ou sm_52) des fichiers créés ?

Correction

Le suffixe *_30* (ou *_52*) correspond à la compute capability : ici 3.0 (ou 5.2).

La compute capability désigne l'ensemble des caractéristiques de l'architecture d'un cœur de calcul (SM = Streaming Multiprocessor). Chaque nouvelle version permet de supporter de nouvelles fonctionnalités.

Par défaut, nvcc compile les sources pour une certaine compute capability en fonction de la version de nvcc. Par exemple, nvcc 12.3 compile par défaut pour une compute capability de 5.2.

La valeur peut également être modifiée via l'option de compilation : `-gencode=arch=compute_61,code=sm_61`. Ou en version courte : `-arch=sm_61`.

Cf la documentation de nvcc pour plus d'infos.

II Multiplication matricielle

Le but de cette partie est de d'implémenter une première version de multiplication de matrices sur GPU, appelé "General Matrix Multiplication" (GEMM).

La mémoire globale est accessible depuis n'importe quel thread d'une grille sur un GPU. C'est la mémoire qui est allouée/libérée grâce aux fonctions `cudaMalloc` et `cudaFree`. Nous allons dans un premier temps utiliser cette mémoire pour écrire un premier kernel de produit matriciel.

La multiplication de deux matrices est un problème qui se prête bien au système de fonctionnement du GPU. Pour vous aider, un code séquentiel écrit en C est à votre disposition, il s'agit du fichier `dgemm.c`. À partir d'ici, veuillez à compléter ce programme afin de déporter les calculs de la multiplication des deux matrices sur un GPU. N'oubliez pas de changer l'extension du fichier (*i.e.* `.c` en `.cu`).

Soient A , B et C trois matrices carrées de tailles $N \times N$, la multiplication de la matrice A par la matrice B stockée dans C peut se définir par l'algorithme suivant :

Listing 1 – Multiplication de matrice CPU en C

```
1 double A[N*N];
2 double B[N*N];
3 double C[N*N];
4
5 for (int i = 0; i < N; i++)
6 {
7     for (int j = 0; j < N; j++)
8     {
9         double tmp = 0.0f
10        for (int k = 0; k < N; k++)
11        {
12            tmp += A[i * N + k] * B[k * N + j];
13        }
14        C[i * N + j] = tmp;
15    }
16 }
```

1 Allocation et transfert de données

Q.4: Lisez l'introduction de la page Wikipedia "Row- and column-major order". Quelle représentation avons-nous choisie dans l'extrait de code précédent ? Sauf mention contraire, nous utiliserons cette représentation dans la suite du TP.

Correction

Row-major.

Une explication est donnée en détail dans l'article qui vous est proposée. En C, CPP et CUDA, les matrices sont organisées par lignes contigües dans la mémoire. (Dans d'autres langages, par exemple en FORTRAN, elles sont organisées par colonnes)

Ainsi, nous faisons le choix de parcourir les matrices A et C en row-major pour profiter des effets de cache possibles. Par exemple, à l'itération 0 de la boucle k, quand le thread charge la valeur de $A[i * N]$, le thread va charger une ligne de cache contenant non seulement $A[i * N]$, mais également les données qui lui sont contigües dans la mémoire, donc $A[i * N + 1]$, $A[i * N + 2]$, etc, selon la taille de la ligne de cache. De cette manière, le chargement des valeurs de A sera plus rapides.

À l'inverse, la matrice B est lue par colonne. Or sa représentation dans la mémoire est en row-major, donc nous ne profiterons pas au mieux des effets de cache.

Q.5: Dans un premier temps, à l'aide de la fonction `cudaMalloc`, allouer trois vecteurs de tailles $N \times N$ sur le GPU : deux vecteurs en entrée pour les opérandes de la multiplication, et un troisième pour stocker le résultat final.

Ici, nous représentons une matrice sous la forme d'un seul vecteur de taille $N \times N$ comme dans le listing 1. L'intérêt est de stocker les matrices de manière contigüe en mémoire et de ne pas avoir à faire des transferts de tableaux à outrance (par exemple un tableau par ligne si l'on stocke une matrice sous forme d'un vecteur de vecteurs).

Correction

On déclare trois vecteurs `d_a`, `d_b` et `d_b` qu'on alloue dans la mémoire globale du GPU avec `cudaMalloc(&ptr, sizeof(double) * N * N)`.

Q.6: Les données étant maintenant allouées, transférez les données initialisées par l'hôte (*i.e.* Host) vers l'accélérateur (*i.e.* Device) en utilisant la fonction `cudaMemcpy`.

Correction

On transfère les matrices A et B vers `d_a` et `d_B` avec `cudaMemcpy(ptr, sizeof(double) * N * N, cudaMemcpyHostToDevice)`.

2 Configuration du kernel

Q.7: Avant de déporter du calcul sur l'accélérateur, il faut configurer la grille sur laquelle vont s'exécuter l'ensemble des threads CUDA. À l'aide des structures de type `dim3`, configurez une grille 2D contenant des blocs 2D de taille 32x32 (*i.e.* 1024 threads par bloc). Cette grille devra contenir suffisamment de threads afin que chacun d'entre eux puisse être en charge du calcul d'un élément de la matrice résultat.

Correction

```

Le bloc de taille 32x32 est déclarée par dim3 blockSize(32, 32);
Pour la grille 2D :
int nb_blocks = N / 32; // Nombre de blocs selon x et y
/* Si N n'est pas multiple de 32, il faut des blocs
supplémentaires pour calculer tous les éléments de la
matrice C */
if ((N % 32) > 0) nb_blocks++;
dim3 gridSize(nb_blocks, nb_blocks);

```

Q.8: L'heure est venue de faire s'exécuter le premier kernel sur un accélérateur de calcul. Écrire un premier kernel, pour l'instant vierge, sur la grille préalablement configurée à la question précédente.

Correction

```

On déclare le kernel avec le mot-clé __global__, par exemple __global__
void mat_mul() {}. On le lance avec les paramètres de la question pré-
cédente : mat_mul<<< gridSize, blockSize >>>();

```

Q.9: En l'état, le kernel s'exécute sur l'accélérateur de calcul mais ne retourne aucun résultat sur la partie hôte. À la suite de l'exécution du kernel, veiller à transférer le résultat qui nous intéresse pour le calcul de la multiplication matricielle.

Correction

```

Utiliser à nouveau la fonction cudaMemcpy dans le sens DeviceToHost
pour transférer les résultats de d_c vers C.

```

3 Écriture du kernel

Q.10: Dans le kernel, à partir des dimensions et des indices relatifs à la grille, calculez l'indice de la ligne et de la colonne de l'élément de C qu'il faudra manipuler. **Ici, utilisez une grille 2D avec la dimension x pour les lignes et la dimension y pour les colonnes.** Faites un schéma de votre grille de blocs sur un brouillon pour vous aider.

Correction

```

Il y a au moins deux manières de procéder.
1 - Soit on calcule directement deux indices globaux différents pour la
ligne et la colonne.
2 - Soit on calcule d'abord l'indice global du thread, puis on en déduit la
ligne et la colonne.
Méthode 1
int row = threadIdx.x + blockDim.x * blockIdx.x;
int col = threadIdx.y + blockDim.y * blockIdx.y;
Méthode 2
int block_id = blockIdx.y * gridDim.x + blockIdx.x;
int tid_in_block = threadIdx.y * blockDim.x + threadIdx.x;
int tid = (blockDim.x * blockDim.y) * block_id +
tid_in_block;
int row = tid % N; // Selon x
int col = tid / N; // Selon y

```

Q.11: Pour un thread donné, effectuez le calcul de la case de C dans le noyau exécuté par le GPU.

Correction

Voir fichier `dgemm_7.c`

4 Analyse des performances

Q.12: Maintenant que le calcul de multiplication matricielle fonctionne, il nous faut analyser les performances de notre application. Le SDK CUDA fournit des fonctions permettant de calculer le temps total passé dans un kernel.

Utiliser ces fonctions et regardez le temps mis par l'application pour calculer le produit de deux matrices et le transfert du résultat sur la partie hôte. Comparez ces performances au noyau s'exécutant uniquement sur CPU, de manière séquentiel.

Indices : le guide de programmation CUDA vous indique la marche à suivre pour se servir des événements CUDA de type `cudaEvent_t`. Ceux-ci doivent d'abord être créés avec `cudaEventCreate`. Ils fonctionnent habituellement par paire : un premier enregistre l'état de départ, un autre l'état d'arrivée. De cette manière, on peut obtenir l'évolution de l'état du GPU, c'est à dire un ensemble de métriques, sur l'ensemble des tâches lancées entre le départ et l'arrivée sur le GPU, notamment le temps écoulé. Pour enregistrer un état, on utilise la fonction `cudaEventRecord`. Cette fonction étant asynchrone, il faudra attendre la fin de l'enregistrement de l'état avec `cudaEventSynchronize`. Pour obtenir la durée écoulée entre deux états, on utilise la fonction `cudaEventElapsedTime`.

Correction

Voir fichier `dgemm.cu`

Q.13: Dans cette question, nous considérons la multiplication de deux matrices carrées 4096×4096 de flottants simple précision (FP32). Nous souhaitons estimer une borne inférieure du temps nécessaire pour effectuer la multiplication sur GPU des matrices.

Estimez

- le nombre d'opérations flottantes
- le volume de données minimal V_r à lire (en MB)
- le volume de données minimal V_w à écrire (en MB)

nécessaire pour calculer le produit matricielle.

Quelle est la bande passante mémoire maximale théorique de votre GPU ? Quelle est le débit d'opérations flottantes maximal théorique de votre GPU ? (cf TP0). En utilisant ces deux données et en considérant une implémentation idéale qui ne transfère que le volume de données minimal $V = V_r + V_w$ depuis ou vers la mémoire globale, calculez

- le temps minimal t_{data} pour effectuer les transferts de données
- le temps minimal $t_{compute}$ pour effectuer les calculs

Calculez $\rho = \frac{t_{compute}}{t_{data}}$. Commentez.

Correction

Pour des flottants FP32, on a $(32 / 8)$ octets de taille de donnée par flottant. Dans le meilleur des cas, on ne lit qu'une fois totalement A et B et on écrit tous les éléments de C.

— $FLOPS = 2 \times 4096^3 = 137 \text{ GFLOPS}$

— $V_r = 2 \times 4096^2 \times (32/8) = 128 \text{ MB}$

— $V_w = 4096^2 * 4 = 64 \text{ MB}$

Donc $V = 192 \text{ MB}$.

Soient BW la bande passante crête du GPU et F son débit crête d'opération flottante pour les opérations FP32. Alors $t_{data} = V/BW$ et $t_{compute} = FLOPS/F$

Il est attendu que ρ soit grand devant 1.

Quadro P5000 :

$BW = 192 \text{ GB/s}$ et $F = 6200 \text{ GFLOP/s}$. D'où $t_{data} = 1 \text{ ms}$ et $t_{compute} = 22 \text{ ms}$. D'où $\rho = 22 > 1$. Donc le kernel est compute-bound.

A100 :

La peak memory bandwidth du A100 est 1555.20 GB/s Sa puissance de calcul crête est 19.5 TFLOPs . Pour ce GPU on a donc $t_{data} = 0.13 \text{ ms}$ et $t_{compute} = 7 \text{ ms}$. Donc $\rho = 54 > 1$. Donc le kernel est compute-bound.

Le kernel est compute-bound tant que ρ est supérieur à 1.

Remarque : on peut refaire les calculs avec des flottants en double-précision FP64. Dans ce cas, le kernel reste compute-bound, mais la puissance de calcul crête du GPU est au moins divisé par deux. Voir détail de l'architecture de votre GPU pour ce dernier point.

Q.14: Calculez le débit d'opérations flottantes de votre kernel. Comparez le au débit d'opération flottante maximal de votre GPU. Commentez.

Correction

Résultat sur un GPU A100-SMX4 40GB avec $N = 4096$: 220.1 GFlops .
Ce qui est peu devant les 19.5 TFLOPs théoriques de ce GPU !
Sur Quadro P5000 : 47.56 GFlops . Là encore loin des 6200 GFlops FP32 théoriques, soit 3100 GFlops FP64 théoriques.

Q.15: L'objectif de cette question est de décrire à la main le pattern d'accès mémoire du kernel naïf. On suppose ici qu'une ligne de cache fait 128 B . Dans le kernel naïf, quel volume de données doit charger un thread pour calculer une valeur de C ? Est-ce que les threads accèdent à des données contiguës en mémoire ?

En considérant le pire cas où chaque lecture ou écriture de données génèrent un cache miss, estimez le trafic mémoire requis pour effectuer le produit matricielle de 2 matrices 4096×4096 .

Refaites le calcul en considérant le cas plus réaliste où le thread ne génère qu'un seul cache miss lorsqu'il accède au premier élément d'un ensemble d'éléments contigus en mémoire, et autant de cache miss que d'éléments accédés depuis un ensemble non contigu.

Commentez par rapport à la taille des matrices.

Correction

Chaque thread doit lire une colonne de B et une ligne de A pour calculer un élément de C. Pour une matrice $N \times N$ de FP64, cela représente au pire autant de lignes de caches que d'éléments à accéder : $V_{worst_case} = (64/8) \times N \times 128$ B, au mieux seulement autant d'éléments que nécessaire : $V_{line} = (64/8) \times N$ B.

Vu le format de stockage de la matrice, une ligne est une donnée contigüe au contraire d'une colonne.

Dans le pire cas, chaque thread génère autant de cache miss que de lecture et d'écriture. Le thread lit deux matrices et écrit dans une autre. On lance autant de threads que d'éléments dans la matrice de résultats C.

Pire cas :

- $V_{thread} = 3 \times V_{worst_case} = 12$ MB
- Trafic mémoire total : $V = 4096^2 V_{thread} = 192$ TB !

Cas plus réaliste :

- $V_{thread} = 3 \times V_{best_case} = 96$ KB
- Trafic mémoire total : $V = 4096^2 V_{thread} = 512$ GB

Une matrice de 4096×4096 FP64 éléments fait 128 MB, donc le pattern d'accès mémoire n'est pas optimal du tout dans notre kernel naïf.

III Optimisation du pattern d'accès à la mémoire globale

Dans cette section, nous allons optimiser le kernel de la partie précédente en changeant le pattern d'accès mémoire de manière à ce que le GPU puisse effectuer des accès mémoires "coalescés" ("global memory coalescing" en anglais).

1 Concept de warp

Nous vous rappelons ici le concept de warp. Lors de l'exécution d'un kernel, les blocs de threads sont assignés aux SM. Au sein d'un même bloc, les threads sont regroupés en paquets de 32 threads appelé "warps". Les warps sont assignés à un warp scheduler (il y en a plusieurs par SM) qui se charge d'exécuter les instructions sur les cœurs du GPU.

Le groupement des threads en warp se fait par *threadId* global consécutif. Par exemple, pour une grille 2D de blocs 2D, l'indice global *threadId* est (cf TD1) :

$$threadId = (blockIdx.y \times gridDim.x + blockIdx.x) \times blockDim.x * blockDim.y + threadIdx.y \times blockDim.x + threadIdx.x$$

Les threads de *threadId* consécutif sont regroupés en warps comme illustré sur la figure suivante.

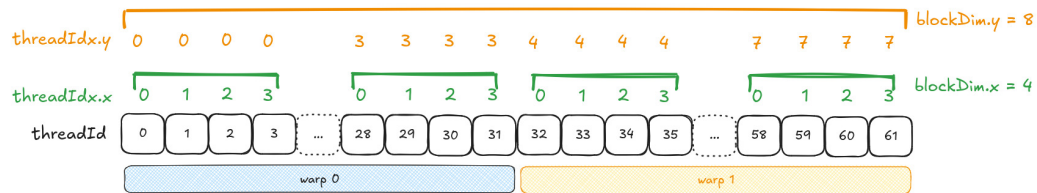


FIGURE 1 – Regroupement des threads en warps. $blockIdx = (0, 0)$

2 Global memory coalescing

Les accès mémoire séquentiel des threads d'un même warp peuvent être regroupés et exécutés comme un seul accès par le hardware. Cela est appelé le "global memory coalescing".

Le GPU supporte des accès mémoire de taille 32B, 64B et 128B.

Si les accès mémoire des threads d'un même warp sont alignés et contigus en mémoire, alors le GPU sera en mesure de regrouper plusieurs accès en une seule transaction.

Sinon, le GPU effectuera autant d'accès mémoire de taille 32B que nécessaire. Mais cela n'optimise pas l'utilisation de la bande passante mémoire.

La figure suivante illustre un exemple où les accès mémoire contigus des threads d'un même warp sont regroupés, permettant à chaque warp de faire 4 accès mémoire de 32B en utilisant seulement 1 load de 128B.

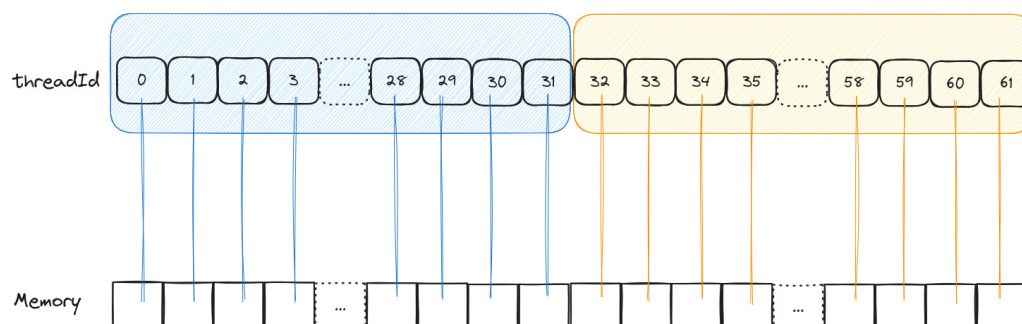


FIGURE 2 – Regroupement des accès à la mémoire globale

3 Accès mémoire du kernel naïf

Vu la représentation de matrices que nous avons choisi, une ligne constitue un ensemble de données contigües en mémoire et une colonne un ensemble de données non contigües en mémoire.

La figure suivante illustre le pattern d'accès mémoire des threads d'un même warp au sein des 3 matrices.

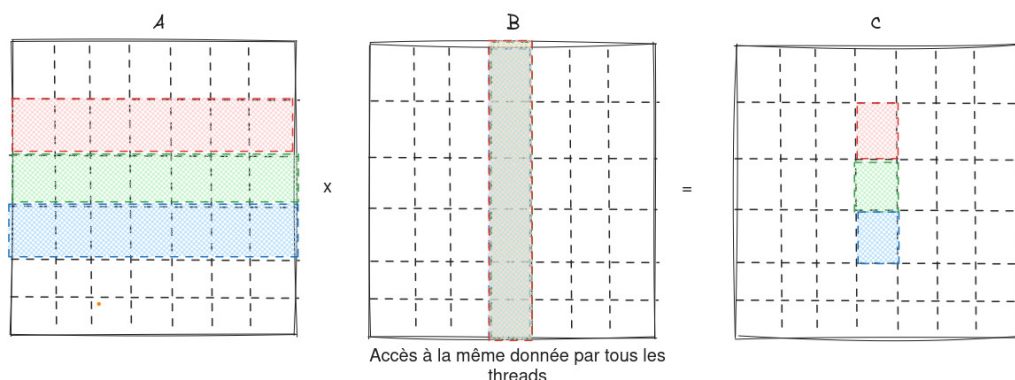


FIGURE 3 – Accès mémoire du kernel naïf

Au sein de A, à chaque tour de boucle, les threads d'un même warp accèdent à une portion de ligne de A. Ces accès ne sont pas contigus et ne peuvent être

coalescés.

Au sein de B, à chaque tour de boucle, tous les threads accèdent à la même donnée qui peut être broadcasté à tous les threads du warp.

Au sein de C, les données écrites ne sont pas contigües en mémoire.

Q.16: Nous allons changer le pattern d'accès mémoire du kernel de manière à bénéficier du global memory coalescing. Cela peut être effectué en changeant la valeur de C calculé par un thread. Nous souhaitons que les threads d'un même warp s'occupent de calculer les valeurs de C situés sur la même ligne. De cette manière les threads d'un même warp liront à chaque tour de boucle

- la même valeur dans A
- des valeurs contigües dans B (car situées sur la même ligne)

Le pattern d'accès mémoire visé est illustré sur la figure suivante.

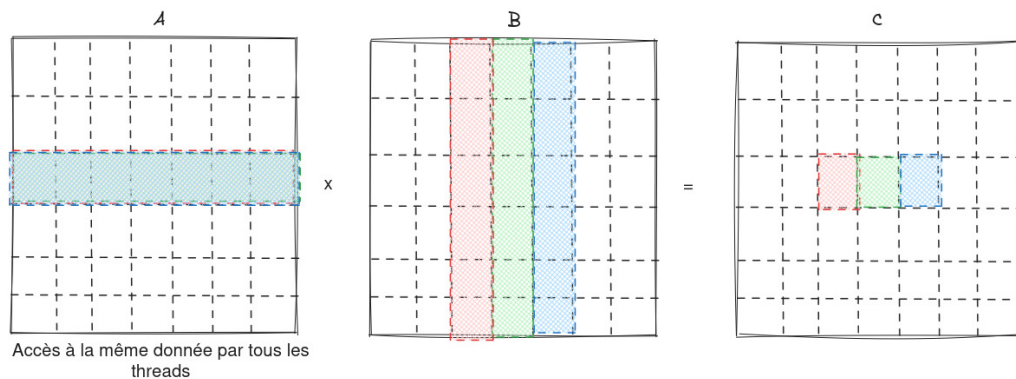


FIGURE 4 – Pattern d'accès mémoire visé

Changez la valeur de C calculé par un thread de manière à obtenir le pattern d'accès mémoire visé.

Correction

Solution 1 : inversez le rôle des lignes et des colonnes.

```
int col = threadIdx.x + blockDim.x * blockIdx.x;
int line = threadIdx.y + blockDim.y * blockIdx.y;
```

Solution 2 :

- utiliser des blocs de threads de dimensions 1D au lieu de 2D en gardant le même nombre de threads au total
- changer les indices de la ligne et de la colonne calculées par un thread.

```
dim3 gridSize(nbBlocks, nbBlocks);
dim3 blockSize(BLOCK_WIDTH * BLOCK_WIDTH);
int line = threadIdx.x / BLOCK_WIDTH + BLOCK_WIDTH *
blockIdx.x;
int col = threadIdx.y % BLOCK_WIDTH + BLOCK_WIDTH *
blockIdx.y;
```

IV Utilisation de la mémoire partagée

La mémoire partagée avec CUDA favorise la coopération entre les threads d'un même bloc. Lorsque plusieurs threads d'un bloc utilise les mêmes données depuis

la mémoire globale, ces données peuvent être chargées dans un espace mémoire partagé entre tous les threads d'un même bloc : c'est la notion de mémoire partagée avec CUDA.

L'objectif ici est d'améliorer l'implémentation que l'on a faite de la multiplication matricielle en utilisant la mémoire partagée. Pour cela, nous allons faire la multiplication de matrices par tuiles. Le schéma 5 en décrit l'idée.

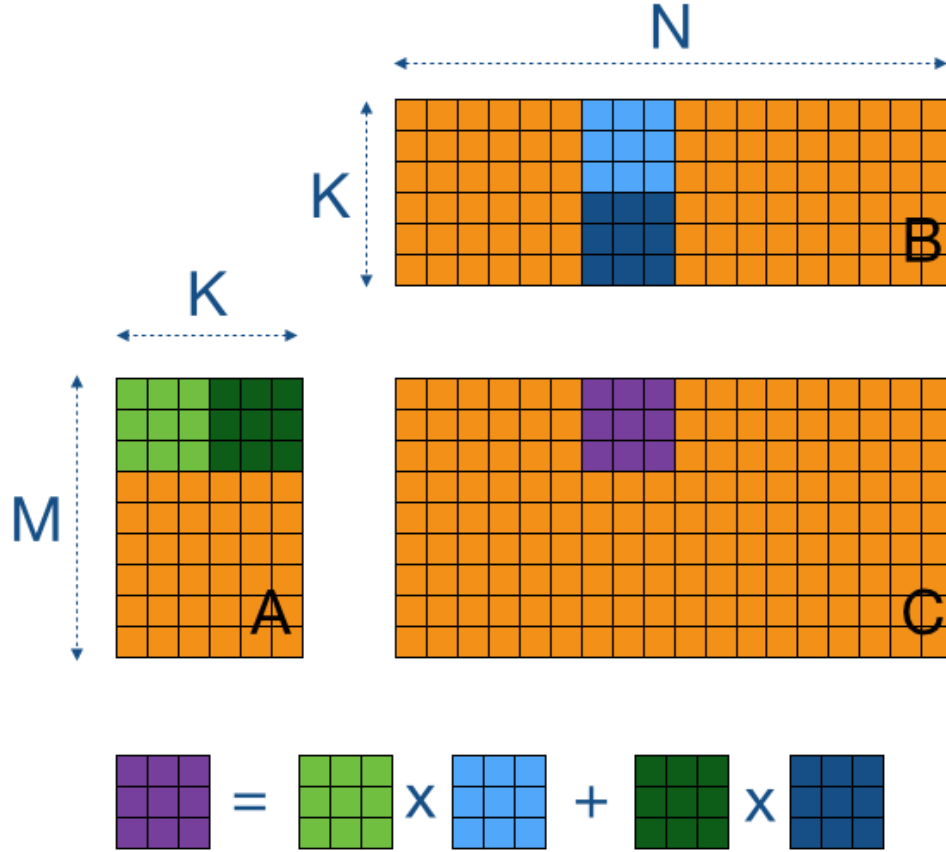


FIGURE 5 – Multiplication de matrices par tuiles

Pour faire le calcul des éléments du bloc violet, nous devons lire les lignes correspondantes de A (en vert) et les colonnes correspondantes de B (en bleu).



FIGURE 6 – Détail de calcul d'un élément de C

Comme le montre la figure 6, pour faire le calcul des éléments du bloc violet, nous réutilisons les éléments des blocs vert et bleu plusieurs fois. C'est pourquoi nous voulons éviter de devoir relire plusieurs fois la même donnée depuis la mémoire globale.

L'idée est de découper les matrices en tuiles de taille aussi grande qu'un bloc. Ces tuiles seront réparties au sein des différents blocs. De même que pour notre

kernel naïf, chaque élément de C ne sera calculé que par un seul thread. L'idée est de profiter de la mémoire partagée pour lire les données des matrices A et B. Ainsi, un bloc de threads ne va lire qu'une seule fois l'entièreté des éléments de A et B dont chacun de ses threads a besoin.

Q.17: Nous allons maintenant travailler sur des tuiles de données, que chaque thread va charger depuis la mémoire globale. Déclarer deux tuiles de données partagées à l'intérieur du kernel. Ces tuiles 2D auront les mêmes dimensions qu'un bloc de la grille configurée pour l'exécution du kernel CUDA.

Correction

```
__shared__ double shared_A[BLOCK_SIZE][BLOCK_SIZE]; ou
__shared__ double shared_A[BLOCK_SIZE * BLOCK_SIZE]
```

Q.18: Dans un second temps, nous allons identifier les données à charger en mémoire partagée pour favoriser la coopération entre les threads d'un même bloc. Chaque thread d'un bloc va charger en mémoire partagée une valeur des deux matrices d'entrée. Comme nous procédons par tuile, écrivez une boucle qui permet, à chaque itération, de charger un élément par thread de chaque matrice d'entrée qui permette de couvrir le chargement de l'ensemble de la matrice.

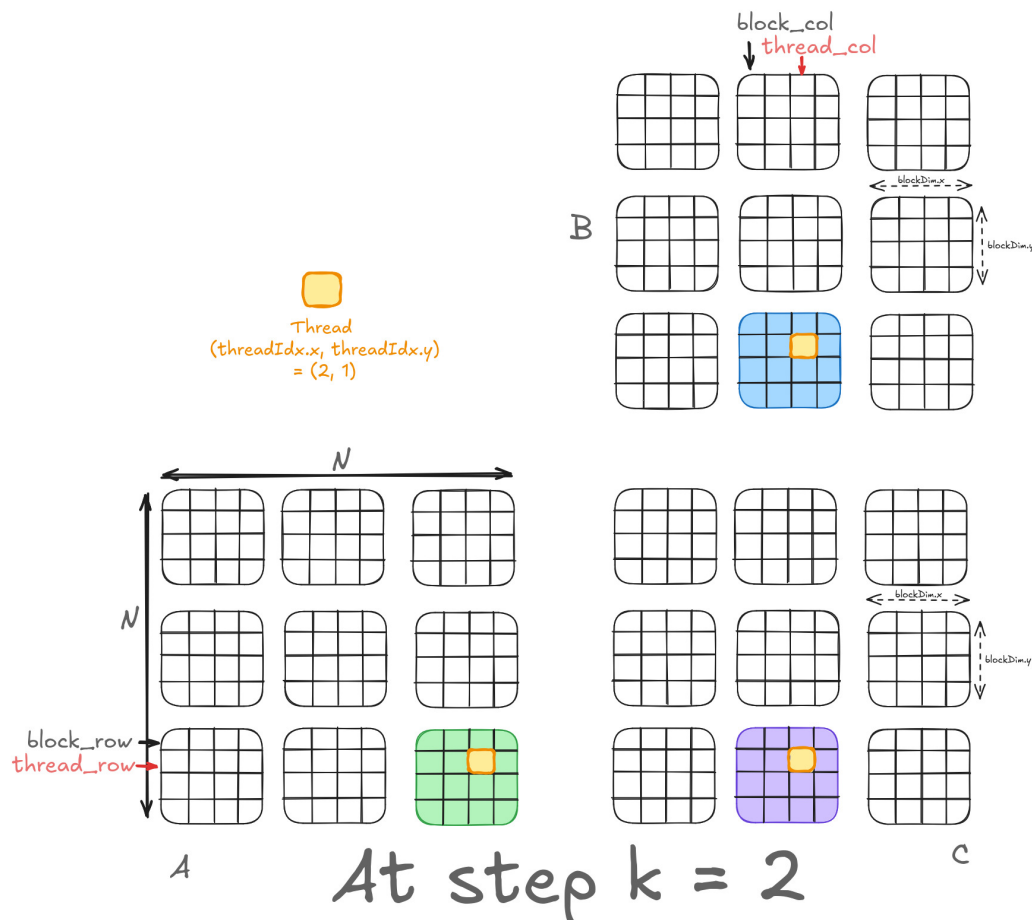


FIGURE 7 – Détail de calcul d'un élément de C

Correction

Voir fichier dgemm.cu

Pour simplifier le code, nous supposons ici que N est multiple de $BLOCK_WIDTH$ ($= 32$). Notons k le nombre allant de 0 à $n_iterations$, le nombre d'itérations pour multiplier par tuiles, c'est-à-dire $n_iterations = N / BLOCK_WIDTH$. Le thread de coordonnées $(0,0)$ dans le bloc va donc charger depuis A l'élément de coordonnées $(block_row, k * BLOCK_WIDTH)$ et depuis B l'élément de coordonnées $(k * BLOCK_WIDTH, block_col)$.

En généralisant à un thread de coordonnées $(threadIdx.x, threadIdx.y)$, on a :

```
int block_row = blockIdx.y * BLOCK_WIDTH;
int thread_row = block_row + threadIdx.y;
int block_col = blockIdx.x * BLOCK_WIDTH;
int thread_col = block_col + threadIdx.x;
int n_iterations = N / BLOCK_WIDTH

for (int k = 0; k < n_iterations ; k++) {
int a_col = k * BLOCK_WIDTH + threadIdx.x;
int b_row = k * BLOCK_WIDTH + threadIdx.y;
/* Write in shared memory */
s_a[threadIdx.y * BLOCK_WIDTH + threadIdx.x] = A[thread_row
* N /* Shift to correct row */
+ a_col] /* Shift to correct colon */
s_b[threadIdx.y * BLOCK_WIDTH + threadIdx.x] = B[b_row * N
/* Shift to correct row */
+ thread_col] /* Shift to correct colon */
}
```

Q.19: Après avoir chargé les données, nous pouvons maintenant passer à la partie calculatoire. À l'aide des données en mémoire partagée, effectuer le produit matriciel sur le bloc courant et accumuler le résultat dans une variable locale. Stockez la valeur finale dans C à la case correspondante.

Correction

Voir fichier dgemm.cu

Avant la boucle for, déclarer et initialiser une valeur locale pour stocker la valeur de C à chaque étape.

```
double value = 0.;
for (k = ...) {
/* Write in shared memory */
(...)
/* Compute "A row * B col" */
for (l = 0; l < BLOCK_WIDTH; l++) {
value += s_A[threadIdx.y * BLOCK_WIDTH + l] * s_B[l *
BLOCK_WIDTH + threadIdx.x];
}
}
/* Store the value in C (in global memory) */
C[thread_row * N + thread_col] = value;
```

Q.20: Exécuter maintenant votre programme et vérifier le résultat par rapport à la version d'avant. À moins que vous n'ayez déjà paré cette éventualité, le résultat n'est pas correct. Pourquoi ? Comment corriger ce problème ?

Correction

Avant de commencer le calcul, il faut s'assurer que tous les threads aient bien fini de charger les données en mémoire partagée. Il faut donc rajouter un appel à la fonction `__syncthreads()` avant de calculer le résultat et avant de charger à nouveau des données.

Voir fichier `dgemm.cu`

```
double value = 0.;
for (k = ...) {
    /* Write in shared memory */
    (...)
    __syncthreads()
    /* Compute "A row * B col" */
    for (l = ...) {...}
    __syncthreads()
}
```

Q.21: Passons maintenant à l'évaluation des performances. Reportez le temps d'exécution de notre noyau optimisé par rapport au programme non optimisé du TP précédent. Quel est le gain obtenu avec cette nouvelle version ? Le gain est-il le même pour des matrices en précision FP32 (float) et en précision FP64 (double) ?

Correction

Tests effectués sur le Cluster Roméo (Nvidia P100, Cuda 10.1), $N = 4096$.

Sans optimisation : 0,642344s

Avec optimisation : 0.063814s

Gain obtenu : Speedup = 10,07.

Sur un GPU A100-SMX4 40GB : 0.033718s pour $N = 4096$ soit 4076.1 GFLOP/s.

Sur un GPU Quadro P5000 (Driver 560.35.03 et CUDA 12.6), avec $N = 4096$:

Pour des FP64, la version naïve se fait en 4.10s, soit 34 GFLOP/s. La version coalescée en 1.31s, soit 105 GFLOP/s. La version par tuiles en 1.20s, soit 115 GFLOP/s, donc environ 3.4 fois plus rapide que la version naïve.

Pour des FP32, la version naïve se fait en 4.47s, soit 31 GFLOP/s. La version coalescée en 0.88s, soit 156 GFLOP/s. La version par tuiles en 0.23s, soit 597 GFLOP/s, donc environ 19 fois plus rapide que la version naïve. Sur ce GPU, il semble qu'on bénéficie bien plus de la mémoire partagée en précision FP32 qu'en précision FP64.